

CONTRACT INTELLIGENCE CHATBOT

System Design Document

Version 1.0 | June 2025

1. Executive Summary

The Contract Intelligence Chatbot is an AI-powered document querying system that enables legal, procurement, and operations teams to interrogate contract documents using natural language. The system combines two complementary retrieval paradigms — Knowledge RAG for direct clause lookup and Agentic RAG for multi-step reasoning tasks such as cross-contract comparison and compliance analysis.

The solution is built on a Streamlit frontend, a FastAPI backend, and a LangGraph orchestration layer that routes each query to the appropriate agent pipeline. FAISS provides low-latency vector search, SQLite persists session history, and OpenAI models power both embeddings and language generation.

2. Objectives and Scope

2.1 Primary Objectives

- Allow users to upload one or more contract documents (PDF) and ask natural language questions about their content.
- Support both simple clause lookup (Knowledge RAG) and complex multi-document reasoning (Agentic RAG).
- Return answers with source citations (document name + page number) so users can verify responses.
- Maintain conversation history per session to enable follow-up questions.

2.2 In Scope

- Multi-document upload with per-document and cross-document query scoping.
- Document ingestion pipeline: PDF parsing, chunking, embedding, and FAISS indexing.
- Knowledge RAG workflow for clause lookup, summaries, payment terms, and renewal dates.
- Agentic RAG workflow for contract comparison, risk identification, and compliance review.
- Session management with persistent chat history.
- Streamlit UI with Upload Screen and Chat Interface Screen.
- FastAPI REST backend with CRUD endpoints for documents and sessions.

2.3 Out of Scope

- Multi-user authentication and role-based access control (Phase 2).
- Cloud-native deployment to Azure (addressed in Deployment Considerations section).
- Support for non-PDF formats (DOCX, XLSX) — Phase 2.
- Real-time collaboration between multiple users on the same session.

3. Technology Stack

Layer	Technology	Purpose	Version
Frontend	Streamlit	Chat UI and document upload screens	1.32+
Backend API	FastAPI	REST endpoints, request validation	0.110+
Orchestration	LangGraph	Agent graph, intent routing, state machine	0.1+
LLM Frameworks	LangChain	Embedding, chunking, retriever abstractions	0.2+
Language Model	OpenAI GPT-4o / Claude	Answer generation, agent reasoning	Latest
Embeddings	text-embedding-3-small	Vector representation of document chunks	OpenAI API
Vector Store	FAISS	Approximate nearest-neighbour search	1.7+
Session Store	SQLite	Chat history persistence	Built-in
Doc Storage	Azure Blob Storage	Raw PDF storage	Azure SDK / azure-storage-blob latest
PDF Parsing	PyPDFLoader (LangChain)	Text extraction from PDF files	Latest
Evaluation	RAGAS	RAG pipeline evaluation — faithfulness, answer relevancy, context precision/recall	Latest

Why FAISS? (Current MVP Scope)

- Designed for a **single-user, single-session** application, making multi-tenancy unnecessary.
- **Runs locally** with no server, infrastructure, or hosting overhead.
- **Open-source and cost-free**, aligning with the low-cost MVP objective.
- Provides **fast in-memory vector search** for small to medium-sized contract collections.
- **Easy to integrate** with LangChain and the existing local file system + SQLite architecture.
- Simple persistence is sufficient since **multi-user access is out of scope**.

Why Not Pinecone / Weaviate / Azure AI Search?

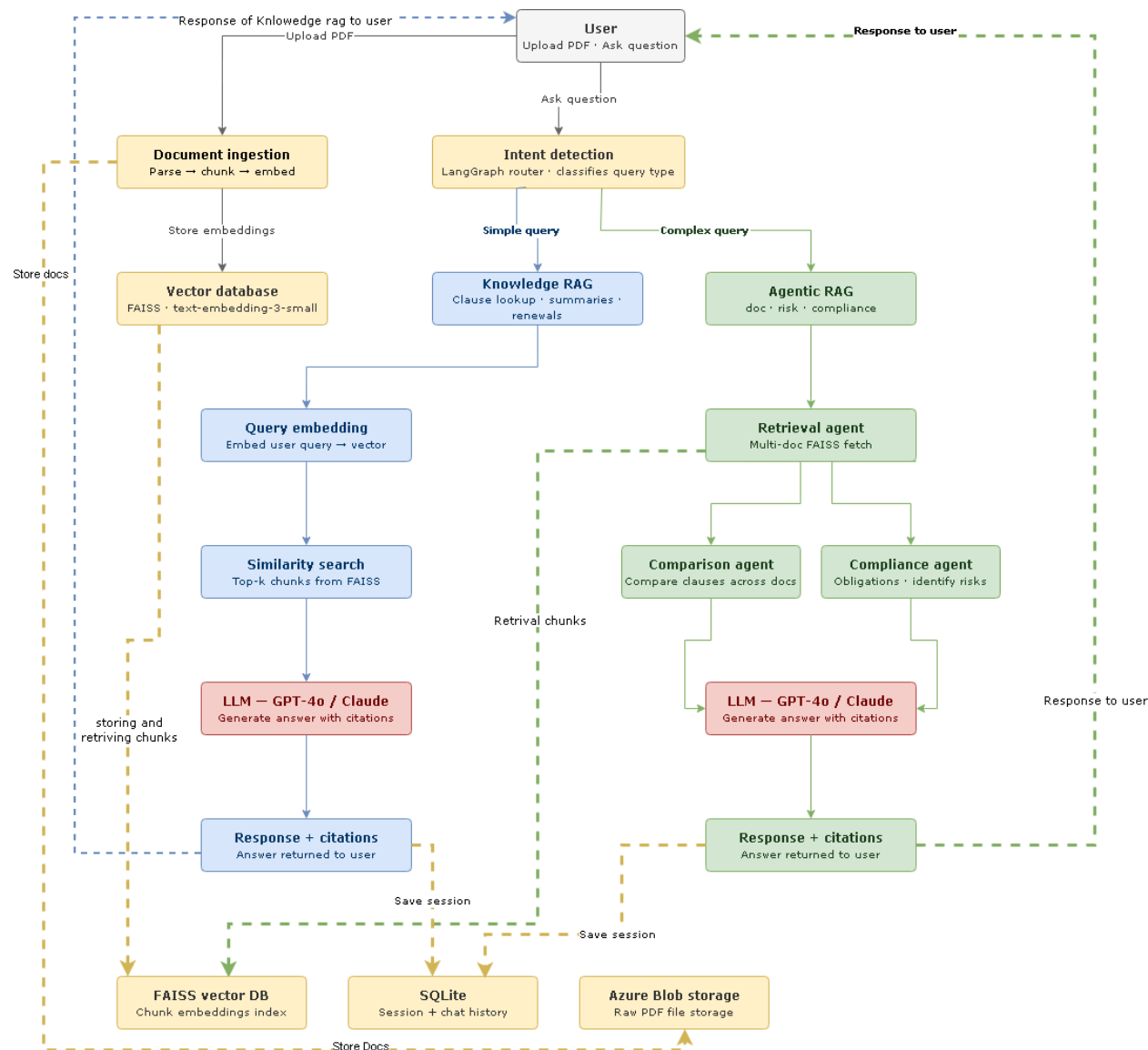
- Built for **multi-user, large-scale, high-availability** deployments, which exceed current requirements.
- Introduce **hosting costs, API management, and operational complexity** without significant benefits for the MVP.
- Depend on external services, whereas the current design follows a **local-first approach**.

Future Migration Strategy

As the system scales to multiple users, migrate to Azure AI Search for enterprise-grade vector search, hybrid retrieval, RBAC, and seamless Azure integration. Consider Weaviate for multi-cloud flexibility or Pinecone for minimal operational overhead. Migration becomes necessary when supporting concurrent users, tenant isolation, incremental indexing, managed backups, or datasets exceeding single-node memory.

4. System Architecture

HIGH level Architecture Diagram



4.1 High-Level Overview

The system is divided into five logical tiers: Frontend, API Gateway, Orchestration, Storage, and LLM. Each tier has a single, well-defined responsibility and communicates only with adjacent tiers.

FRONTEND (Streamlit)

Two-screen Streamlit application. Screen 1 handles document ingestion and status tracking. Screen 2 provides the conversational chat interface with scope selection.

API GATEWAY (FastAPI)

All frontend requests pass through the FastAPI backend. The API validates inputs, manages file storage, coordinates the ingestion pipeline, and forwards chat requests to the LangGraph orchestrator.

ORCHESTRATION (LangGraph)

The LangGraph Intent Router inspects each query and routes it to either the Knowledge RAG pipeline (simple lookups) or the Agentic RAG pipeline (complex reasoning). A shared Retrieval Agent fetches relevant chunks from FAISS for both pipelines.

STORAGE

Three storage components operate in this tier: the FAISS vector database (chunk embeddings), SQLite (session and message history), and the Azure Blob storage (raw PDFs). Each is accessed only via the backend; the frontend never touches storage directly.

LLM

GPT-4o (or Claude) receives a structured prompt containing retrieved context chunks and the user query. The Response Agent post-processes the LLM output to attach source citations before returning the answer to the frontend.

5. Frontend Design (Streamlit)

5.1 Screen 1 — Document Upload

This screen is the entry point for new users. It allows uploading one or more PDF contracts and tracks the status of each document through the ingestion pipeline.

Components

- Multi-file PDF uploader widget.
- Per-document status indicator: Uploading → Parsing → Chunking → Embedding → Ready.
- Document list table showing: document ID, filename, page count, chunk count, and upload timestamp.
- Delete button per document (calls DELETE /document/{id} and removes embeddings from FAISS).

State Management

- st.session_state stores the active document list and their processing statuses.
- Polling or callback from the backend updates status indicators in real time.

5.2 Screen 2 — Chat Interface

The chat screen provides the primary user experience. It combines a document scope selector, a conversation history panel, a query input box, and a response display with citations.

Components

- Scope selector (radio group): All Documents | Contract_A.pdf | NDA.pdf | MSA.pdf.
- Scrollable message history panel showing alternating user/assistant bubbles.
- Text input box with a Submit button and a New Session button.
- Each assistant response includes an expandable Citations section listing source document and page number.
- Session selector sidebar: list of previous sessions with timestamps; click to restore.

Session Behaviour

- Each browser session initialises a new session_id via POST /sessions.
 - On page load, the sidebar fetches GET /sessions to populate the history list.
 - Selecting a past session fetches GET /sessions/{id}/messages and replays it into the history panel.
-

